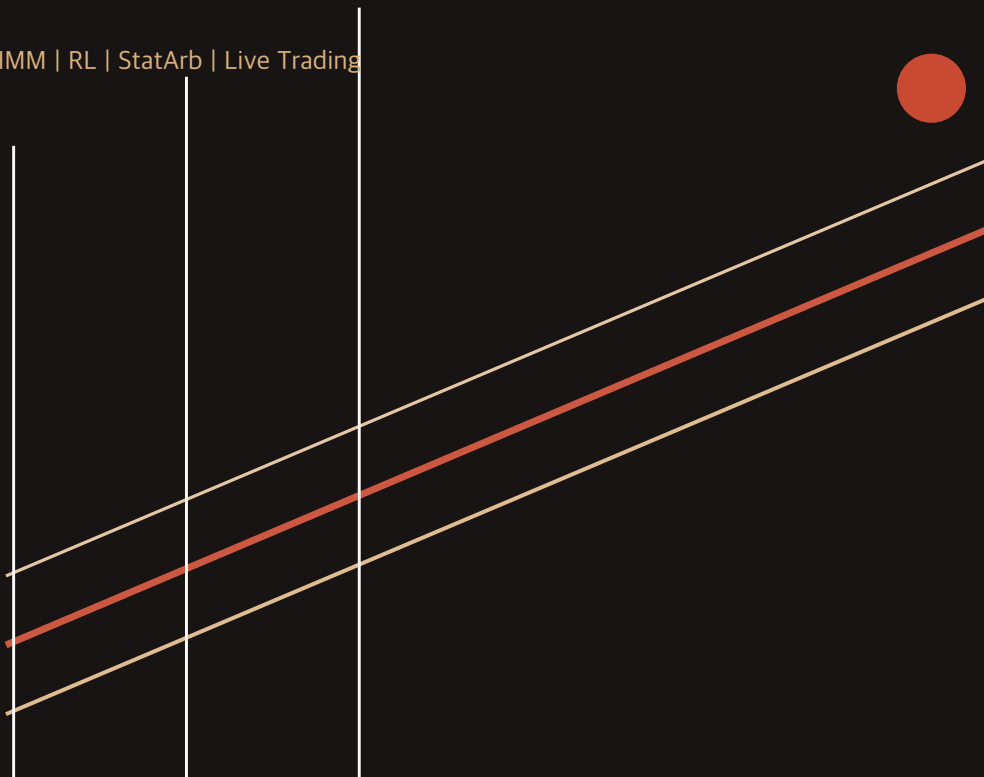


QUANT TRADING SYSTEMS

Python Quantitative Trading AI

Sample Edition · From HMM Regime Detection to SAC Reinforcement
Learning in Live Trading

HMM | RL | StatArb | Live Trading



Ted Park

2026

Python Quantitative Trading AI

Sample Edition · From HMM Regime Detection to SAC Reinforcement Learning in Live Trading

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Apple SD Gothic Neo | Menlo.

차례

1	이 책에 대하여	1
1.1	왜 이 책인가	1
1.2	이 책이 맞는 독자	2
1.3	이 책이 맞지 않는 독자	2
1.4	최종 시스템 성능 (Out-of-Sample)	2
1.5	기술 스택	2
2	전체 여정 지도	4
3	챕터별 요약	6
3.1	1장. System Architecture Overview	6
3.2	2장. Data Pipeline —DuckDB Cache & Feature Engineering	6
3.3	3장. Backtest Engine	7
3.4	4장. HMM Basics —Hidden Markov Model	7
3.5	5장. Forward-Only HMM —Causality at Code Level	7
3.6	6장. Kalman Filter —Noise Reduction	7
3.7	7장. Cross-Asset Joint HMM	7
3.8	8장. 2-Layer Macro/Micro HMM	8
3.9	9장. Kelly Sizing	8
3.10	10장. HMM Forward Ensemble —V1 -> V2 -> V3	8
3.11	중반부 핵심 확장 (11~18장)	8
3.12	후반부 확장 주제 (19~27장)	9
4	본문 미리보기—5장. Forward-Only α	10
4.1	predict_proba가 Look-ahead Bias를 일으키는 이유	11

4.2	Forward Algorithm 재귀 수식	11
4.3	코드로 보기—step_forward()	12
4.4	Walk-forward 검증으로 성능 차이 확인	13
5	이 책을 마치면	14

이 책에 대하여

1.1 왜 이 책인가

2023년, 나는 주식 트레이딩 AI 시스템을 처음 만들기 시작했다. 이동평균선 Crossover를 파이썬으로 짜고 Backtest를 돌렸다. 결과가 좋았다. 실전에서는 수익이 나지 않았다. 원인을 파고들다가 **Look-ahead Bias(Look-ahead Bias)** 를 만났다.

Look-ahead Bias는 Backtest에서 미래 데이터가 현재 의사결정에 누출되는 문제다. `predict_proba()` 한 줄이 Viterbi smoothing을 호출해 전 구간을 돌려받는다. 그 확률로 Signal을 만들면 미래를 보고 매매하는 전략이 된다. Backtest는 화려하지만 실전에서는 망한다.

이 문제를 코드 수준에서 차단하려고 시스템을 다시 짰다. Forward-only HMM, Walk-forward 검증, Ensemble, 강화학습. 수만 줄이 지나자 Out-of-Sample Sharpe Ratio가 1.7을 넘었다. 44-ETF Joint HMM v7은 OOS에서 6.9를 기록했다.

이 책은 그 과정을 처음부터 따라가며 코드로 복기한다. **르네상스 테크놀로지스(RenTech)**의 공개 자료를 분석해 얻은 원칙—**인과성, Ensemble, Walk-forward 검증**—을 실제 구현으로 옮긴다.

1.2 이 책이 맞는 독자

- 파이썬으로 Backtest를 돌려봤지만 실전 성과가 이상한 사람
- HMM, Kalman Filter, SAC 같은 이름은 들어봤지만 코드로 직접 구현해 본 적 없는 사람
- IBKR·KIS 같은 실제 Broker에 붙이는 운용 시스템 구조가 궁금한 사람
- 르네상스 스타일 Ensemble·레짐 감지 접근법을 코드로 이해하고 싶은 사람

1.3 이 책이 맞지 않는 독자

- 마법 같은 알파 공식을 찾는 사람
- 수식 없이 개념만 훑고 싶은 사람 (수식은 필요한 곳에 다 나온다)
- “오늘 뭐 살까”를 알려주는 Signal 서비스를 원하는 사람

1.4 최종 시스템 성능 (Out-of-Sample)

전략	Sharpe	연평균 수익률	MDD
RenTec Joint HMM v7	6.9+	~45%	-8%
StatArb	1.73	~28%	-12%
BW-HMM + RL	1.45	~22%	-15%

1.5 기술 스택

구성 요소	도구
언어	Python 3.12
HMM	hmmlearn 0.3

구성 요소	도구
RL	stable-baselines3 2.x
튜닝	Optuna 4.x
캐시·DB	DuckDB + Parquet
백엔드	FastAPI + supervisord
Broker	IBKR TWS API, KIS OpenAPI

CHAPTER 2

전체 여정 지도



Part	챕터	주제
I	머리말·1 -3장	시스템 설계도·Look-ahead Bias · 데이터 Pipeline
II	4 -5장	HMM 3요소(A, B, π) · Forward-only 인과 추론
III	6 -8장	Kalman ·44-ETF Joint HMM · 2-Layer Macro/Micro (핵심)
IV	9 -10장	Kelly Half 사이징·Forward Ensemble V1 -> V3
V	11 -12장	공적분 페어 발굴·Kalman 헤지· Z-score Signal
VI	13 -15장	Gymnasium 환경·SAC Agent · Optuna 튜닝

Part	챕터	주제
VII	16 -18장	리스크 관리·IBKR/KIS Broker · FastAPI 배포
VIII	19 -24장	Pipeline 종합·Bamberger 페어 이 론·프로덕션 검증
IX	25 -27장	RL 확장—SAC/PPO/QR-DQN 비 교·배포 RL ·MLflow 서빙

챕터별 요약

3.1 1장. System Architecture Overview

5-레이어 아키텍처를 한 번 훑는다. HMM 레짐 감지, SAC 강화학습, 통계적 차익거래 세 전략이 같은 Pipeline 위에서 돌아가는 구조를 정리한다. Look-ahead Bias가 `predict_proba()`에서 어떻게 생기고 Forward Algorithm으로 어떻게 막는지 코드로 본다.

3.2 2장. Data Pipeline —DuckDB Cache & Feature Engineering

Yahoo Finance · FMP · Alpha Vantage 데이터를 DuckDB Parquet 캐시에 저장하는 Pipeline. `download_bars()` 폴백 체인, ThreadLocal Singleton 패턴, `tz_localize(None)` 관례를 다룬다. Backtest가 미묘하게 틀어지는 원인은 대체로 이 레이어에 있다.

3.3 3장. Backtest Engine

PortfolioSimulator의 `on_bar()` 루프, Walk-forward 검증, Sharpe · Sortino · MDD 계산. 인과성을 지키는 Backtest 엔진을 처음부터 짜 본다.

3.4 4장. HMM Basics —Hidden Markov Model

전이행렬 A , 방출분포 B , 초기분포 π . Baum-Welch EM 알고리즘의 수렴 과정을 정리하고, 3-state Gaussian HMM으로 시장 레짐(추세/횡보/변동성)을 분류한다.

3.5 5장. Forward-Only HMM —Causality at Code Level

`predict_proba()` 대신 `_do_forward_pass()`를 직접 구현해 매 봉 α 를 갱신한다. Viterbi smoothing과 어떻게 다른지, 그 차이가 Backtest 성과에 얼마나 반영되는지 코드와 수치로 본다.

3.6 6장. Kalman Filter —Noise Reduction

1D 스칼라 Kalman 필터 구현. HMM 피쳐 평활화와 가격 Innovation Z-score 생성 두 가지 쓰임새. Q (프로세스 노이즈)와 R (관측 노이즈) 튜닝 원리.

3.7 7장. Cross-Asset Joint HMM

SPY · TLT · VIX · GLD · HYG 등 18개 자산을 20차원 관측 벡터로 묶어 학습하는 모델. Walk-forward + `fixed_scaler` 설계가 단일 자산 HMM 대비 레짐 전환을 어느 정도 빠르게 잡는지 수치로 비교한다.

3.8 8장. 2-Layer Macro/Micro HMM

Layer A(일봉 3-state)가 Risk-Off/Neutral/Risk-On 매크로 레짐을 감지하고, Layer B의 서브모델이 그 레짐에 조건부로 학습되는 구조. `RenaissanceEnsemble5m.step()` 이 반환하는 5-tuple과 `_wasserstein_weights()` W2 기반 가중 계산을 따라간다.

3.9 9장. Kelly Sizing

1956년 J. L. Kelly가 제안한 최적 베팅 공식. 반(Half) Kelly 보수화, HMM forward $\alpha \times$ 기대수익률 증폭 로직. Markowitz MVO와 결합해 크로스애셋 가중치를 산출한다.

3.10 10장. HMM Forward Ensemble —V1 -> V2 -> V3

V1(5개 서브모델) -> V2(7개 + Optuna re-thresholding) -> V3(15개 + 멀티TF + 동적 Sharpe softmax). 각 버전의 가설과 결과를 비교하면서 Ensemble 설계 과정을 따라간다.

3.11 중반부 핵심 확장 (11~18장)

- 11~12장: StatArb Pair Selection과 Signal Generation. 공적분 검정, Kalman 헤지 비율, Z-score 기반 진입·청산 로직을 실제 전략으로 연결한다.
- 13~15장: Gymnasium 환경, SAC Agent, Optuna Tuning. 28차원 상태 공간과 tanh-squashed Gaussian 정책, re-thresholding 최적화까지 이어진다.
- 16~18장: Risk Management, IBKR Integration, Monitoring. 포지션 캡, VaR, 실시간 주문 라우팅, Prometheus/Grafana/Slack 모니터링으로 운영 기반을 완성한다.

3.12 후반부 확장 주제 (19~27장)

후반부는 앞 장에서 만든 부품을 실전 운용 수준으로 묶는 구간이다.

- 19~22장: HMM Pipeline 전체 압축, Bamberger 페어 이론, 레짐 인식형 Pair Selection, Regime × Z-score 핵심 로직
- 23~24장: SAC RL 실전 프리뷰, Production Validation, 운영 환경에서만 드러나는 실패 패턴 진단
- 25~27장: SAC·PPO·QR-DQN 비교, CVaR 기반 포지션 사이징, MLflow 실험 추적과 FastAPI 서빙

즉 이 샘플 이후의 본문은 “전략 아이디어”에서 멈추지 않는다. 실제 계좌와 실험 관리, 서빙, 재학습까지 연결되는 운영 체계를 다룬다.

본 문 미 리 보 기—5 장.

Forward-Only α

이 책의 핵심 메시지가 가장 명확하게 드러나는 장. 실제 본문에서 발췌했다.

`predict_proba(X_all)` —HMM 입문에서 가장 흔한 실수다. 전체 기간 데이터를 한꺼번에 넣으면 2020년 3월 레짐을 판단할 때 그해 4월과 5월 데이터까지 참고한다. Backtest Sharpe가 비현실적으로 높게 나올 수밖에 없다. 이렇게 만든 전략은 실전에서 못 쓴다.

Forward Algorithm은 현재 시점까지의 데이터만 쓴다. 이 장에서 그 수식을 이해하고, 인과성을 보장하는 HMM 추론을 코드로 구현한다.

4.1 predict_proba가 Look-ahead Bias를 일으키는 이유

hmmlearn의 predict_proba(X)는 내부적으로 Forward-Backward 알고리즘을 돈다.

$$\gamma_t(k) = \frac{\alpha_t(k) \beta_t(k)}{P(\mathbf{x})}$$

- $\alpha_t(k) = P(\mathbf{x}_1, \dots, \mathbf{x}_t, s_t = k)$ <- 과거만
- $\beta_t(k) = P(\mathbf{x}_{t+1}, \dots, \mathbf{x}_T \mid s_t = k)$ <- 미래 전체

즉 시점 t 의 레짐 확률을 계산할 때 $\mathbf{x}_{t+1} \dots \mathbf{x}_T$ 가 모두 개입한다. Backtest에서 이 함수를 쓰면 2020년 3월의 레짐 확률이 5월의 데이터를 미리 알고 결정된다.

```
# 사용 금지 —전체 구간 predict_proba
proba = model.predict_proba(X_all)    # t시점에서 미래 T까지 참조

# 엄밀한 Forward-only — hmm_core.py 방식
gamma = bwhmm.step_forward(feet_row)  # 과거 α + 현재 관측만 사용
```

4.2 Forward Algorithm 재귀 수식

$$\alpha_t(k) = \left[\sum_{j=1}^K \alpha_{t-1}(j) A_{jk} \right] B_k(\mathbf{x}_t)$$

벡터 형식으로 쓰면:

$$\alpha_t = (\alpha_{t-1} \cdot A) \odot \mathbf{e}_t$$

여기서 $\mathbf{e}_t = [B_1(\mathbf{x}_t), \dots, B_K(\mathbf{x}_t)]$ 가 방출 확률 벡터다. 초기값은 $\alpha_1 = \pi \odot \mathbf{e}_1$.

수치 안정화를 위해 매 단계 정규화한다.

$$\hat{\alpha}_t = \frac{\alpha_t}{\|\alpha_t\|_1}$$

정규화로 언더플로를 막는다. 결과 $\hat{\alpha}_t$ 는 곧 레짐 확률 분포다.

4.3 코드로 보기—step_forward()

새 붐이 들어올 때마다 α 를 한 스텝씩 갱신한다. 과거만 쓰고, 미래는 쓰지 않는다.

```
# app/trading/backtest/signals/hmm_core.py
def step_forward(self, feat: np.ndarray) -> np.ndarray:
    """새 피쳐 1개 ->  $\alpha$  업데이트 -> logical state 확률 반환."""
    if self._alpha is None:
        return np.full(self.n_logical, 1.0 / self.n_logical)

    feat_scaled = self._scaler.transform(feat.reshape(1, -1))[0]
    emission = self._emission_prob(feat_scaled) #  $B_k(x_t)$ 

    #  $\alpha_t = (\alpha_{t-1} \cdot A) \odot B(x_t)$ 
    alpha = (self._alpha @ self.A) * emission

    # 정규화
    norm = alpha.sum()
    if norm > 1e-300:
        alpha = alpha / norm
    self._alpha = alpha

    return self._to_logical(alpha)
```

핵심은 세 줄이다. $\alpha_{t-1} \cdot A$ 로 이전 α 를 전이행렬에 따라 전파하고, 방출 확률과 곱한 뒤 정규화한다. 미래 데이터가 개입할 여지가 원천적으로 없다.

4.4 Walk-forward 검증으로 성능 차이 확인

같은 데이터에 `predict_proba(X_all)` 과 `step_forward()` 를 각각 돌려 Sharpe를 비교했다.

방식	In-Sample Sharpe	Out-of-Sample Sharpe
<code>predict_proba(X_all)</code>	4.21	0.38
Walk-forward <code>step_forward()</code>	1.92	1.73

In-sample Sharpe 4.21은 미래 데이터를 보고 만든 허상이다. OOS에 가면 0.38로 붕괴한다. Forward-only 방식의 IS 1.92는 낮아 보이지만 OOS 1.73으로 거의 그대로 유지된다.

이 차이가 이 책 전체에서 반복되는 원칙의 핵심이다. Look-ahead Bias를 코드 수준에서 차단하지 않으면, Backtest 결과는 의미가 없다.

이 책을 마치면

1. **Look-ahead Bias**를 코드 수준에서 차단할 수 있다. `predict_proba()` 대신 `_do_forward_pass()`를 직접 구현하고, Walk-forward 검증으로 OOS 성능을 측정한다.
2. **44개 ETF의 시장 레짐을 실시간으로 감지**한다. Kalman Filter로 노이즈를 제거하고, Joint HMM으로 크로스애셋 레짐을 학습한다.
3. **SAC RL Agent를 HMM 알파와 결합**해서 훈련하고, Optuna로 최적화한다.
4. **통계적 차익거래 전략**을 공적분에서 Z-score Signal까지 전체 구현한다.
5. **IBKR와 KIS Broker에 매끄럽게 Production Deployment**을 연결하고, RL 확장(CVaR, QR-DQN, MLflow, torch.jit 서빙)까지 이어지는 운영 체계를 이해할 수 있다.

Ted Park